

CODEMAP · ARCHITECTURE & LEARNING MAP

Verilog LCD Driver for Spartan-6

16×2 HD44780 character-LCD driver — RTL, self-checking testbench, ISE constraints

Repository	github.com/Sallu-k/Verilog-LCD-Driver-Spartan6_FPGA
Commit	c81ea91 (main)
Author	Salsabeel Kobattey
License	MIT
Documented	2026-10-01
Source	Generated from CODEMAP.md (same content)

Confidence labels: **VERIFIED** (read in source) · **DOCUMENTED** (stated in README/comments only) · **INFERRED** (reasoned from source, not tested) · **UNKNOWN**.

1. Project Identity

Field	Value
Name	Verilog-LCD-Driver-Spartan6_FPGA
Purpose	Verilog RTL that initialises a 16×2 HD44780 character LCD and writes two fixed lines of text.
Problem solved	Drive an HD44780 LCD from an FPGA with no CPU, using fixed delays derived from the datasheet instead of polling the busy flag.
Type	FPGA / digital hardware design (RTL + testbench + constraints)
Status	Simulation passes (DOCUMENTED). Corrected RTL not re-run on hardware (DOCUMENTED).
Technologies	Verilog-2005 (TB compiled with <code>-g2012</code>), Icarus Verilog, GTKWave, Xilinx ISE
Hardware	Xilinx Spartan-6 FPGA board, HD44780-compatible 16×2 LCD
Author	Salsabeel Kobattey (LICENSE, git log). Solo 5th-semester project (README)
Repo	github.com/Sallu-k/Verilog-LCD-Driver-Spartan6_FPGA · branch <code>main</code> · commit <code>c81ea91</code>
Documented	2026-10-01

2. Project Overview

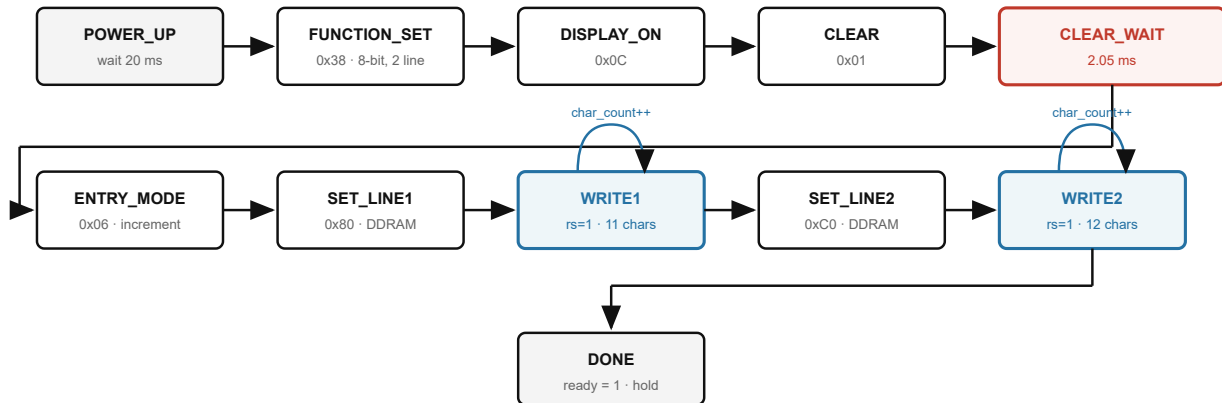
A single Verilog module, `lcd_controller`, drives an HD44780 LCD in 8-bit, write-only mode.

- **Input:** the board clock (`clk`, default 50 MHz) and an active-high reset (`rst`).
- **Processing:** a clock divider makes a one-cycle `tick` every `EN_US` μ s (default 50 μ s). On each tick, an 11-state FSM moves one step. It waits about 20 ms for power-on, then sends Function Set (0x38), Display On (0x0C) and Clear (0x01). After that comes a separate ~2 ms wait (`CLEAR_WAIT`), then Entry Mode (0x06). It then sets the DDRAM address to line 1 (0x80), streams 11 ASCII bytes, sets line 2 (0xC0), and streams 12 bytes.
- **Output:** the LCD bus (`rs`, `rw=0`, `en`, `data[7:0]`) and a `ready` flag that goes high in `S_DONE`.
- **Message text:** hard-coded in a ROM initialised by an `initial` block: "SALSABEEL K" / "SPARTAN6 LCD".

The busy flag is never read (`rw` is tied low), so correct behaviour depends entirely on the delays being long enough. The repo also contains a self-checking Icarus testbench, a Makefile for simulation, and an ISE UCF constraints **template** whose pin LOCs are placeholders. The project has no software runtime, no external services and no storage. Provenance (DOCUMENTED): the design was adapted from an unidentified open-source HD44780 driver. The original `.v` was lost; this file is a reconstruction plus a fix (the missing Clear Display delay).

lcd_controller — FSM

Every transition advances on one EN strobe: byte presented, EN high one settle window, EN low. The HD44780 latches on the falling edge.



 CLEAR_WAIT exists because Clear Display takes 1.52 ms — roughly 40× every other command. Issue the next command too early and it is ignored.

 Write states hold rs=1 and loop on themselves, streaming one ASCII byte per EN strobe until char_count reaches the end of the message.

RW is tied low, so the busy flag is never polled and every wait above is a fixed delay sized from the datasheet.

Figure 1 — lcd_controller FSM (docs/images/fsm_diagram.svg; matches S_* states in rtl/lcd_controller.v)

3. Directory Map

```
Verilog-LCD-Driver-Spartan6_FPGA/
├─ rtl/lcd_controller.v           # the design (only synthesizable file)
├─ tb/lcd_controller_tb.v        # self-checking testbench
├─ constraints/lcd_controller.ucf # ISE pin/timing constraints – TEMPLATE
├─ Makefile                      # sim / wave / clean targets (Icarus)
├─ docs/images/                 # FSM diagram (svg/png), board photos, ISE waveform
├─ README.md
├─ LICENSE                      # MIT
└─ CODEMAP.md / CODEMAP.pdf      # this map
```

- `rtl/`: the synthesizable design, a single module.
- `tb/`: simulation-only verification.
- `constraints/`: Xilinx UCF used for ISE implementation. Every LOC must be replaced before use.
- `docs/images/`: evidence only (photos from the *original* hardware run, the ISE waveform, the FSM diagram). Nothing here is generated by the build.

4. Component Map

Component: Tick Generator (`tick_generator`)

- **Purpose:** divides `clk` into a single-cycle `tick` strobe every `EN_US` μ s.
- **Files:** `rtl/lcd_controller.v` (lines ~40–48)
- **Inputs:** `clk`, `rst`. **Outputs:** `tick`.
- **Depends on:** parameters `CLK_HZ`, `EN_US`. **Used by:** `lcd_fsm`.
- **Behaviour:** `TICKS = (CLK_HZ/1_000_000)*EN_US`. The counter `div` is sized with `$clog2(TICKS+1)`. `tick` is registered and is high for one clock when `div == TICKS-1`.
- **Evidence:** VERIFIED.

Component: LCD FSM (`lcd_fsm`)

- **Purpose:** carries out the HD44780 init sequence and the character writes.
- **Files:** `rtl/lcd_controller.v` (FSM always block, task pulse)
- **Inputs:** `tick`, `rst`, `message_rom`. **Outputs:** `rs`, `en`, `data[7:0]`, `ready`; `rw` is constant 0.
- **Depends on:** `tick_generator`, `message_rom`. **Used by:** the top level / the board (via UCF), `sim_verification`.
- **Behaviour:** 11 states (`S_POWER_UP` ... `S_DONE`, 4-bit encoding). Every byte takes 2 ticks: phase 0 drives the byte and sets `EN=1`, and phase 1 sets `EN=0` and advances. The LCD latches on the falling edge of `EN`. `wait_ctr` (16-bit) is shared by `S_POWER_UP` and `S_CLEAR_WAIT`. `char_count` (4-bit) indexes the ROM.
- **Config:** `BOOT_TICKS = 20000/EN_US + 1`, `CLEAR_TICKS = 2000/EN_US + 1`.
- **Evidence:** VERIFIED.

Component: Message ROM (`message_rom`)

- **Purpose:** holds the two display strings.
- **Files:** `rtl/lcd_controller.v` (`line1`, `line2`, `LEN1 = 11`, `LEN2 = 12`)
- **Outputs:** ASCII bytes to `lcd_fsm`.
- **Behaviour:** `reg` arrays filled by an `initial` block. To change the text, edit the arrays **and** `LEN1` / `LEN2`, keeping each at 16 or less (16 columns; `char_count` is 4 bits).
- **Evidence:** VERIFIED. The 16-character limit is INFERRED from the display width and the counter width.

Component: Simulation Verification (`sim_verification`)

- **Purpose:** checks the byte stream and the Clear Display delay.
- **Files:** `tb/lcd_controller_tb.v`, `Makefile`
- **Inputs:** a DUT instantiated with `CLK_HZ = 1_000_000`, `EN_US = 2` (compressed timing) and a 10 ns sim clock.
- **Outputs:** console PASS/FAIL and `lcd_sim.vcd`.
- **Behaviour:** on `negedge en`, compares `{rs, data}` with `exp_data / exp_rs` (`N = 28`). After `ready` it checks `dut.CLEAR_TICKS * dut.EN_US >= 1520 μs`.
- **Evidence:** VERIFIED. $\triangle$ Covers only 28 of the 29 bytes the DUT writes (see §14).

Component: FPGA Integration (`fpga_integration`)

- **Purpose:** maps ports to pins and constrains the clock for Xilinx ISE.
- **Files:** `constraints/lcd_controller.ucf`
- **Behaviour:** sets the clock PERIOD to 20 ns (50 MHz). All I/O is LVCMOS33. LCD outputs use `DRIVE = 8` and `SLEW = SLOW`, and `rst` uses `PULLDOWN`. `ready` $\rightarrow$ LED is commented out.
- **Evidence:** VERIFIED. **All LOC values are placeholders** (P123–P143).

5. File Responsibility Map

File	Component	Responsibility	Important symbols
<code>rtl/lcd_controller.v</code>	<code>tick_generator</code> , <code>lcd_fsm</code> , <code>message_rom</code>	Entire design	<code>lcd_controller</code> , <code>CLK_HZ</code> , <code>EN_US</code> , <code>TICKS</code> , <code>tick</code> , <code>BOOT_TICKS</code> , <code>CLEAR_TICKS</code> , <code>CMD_*</code> , <code>S_*</code> , <code>pulse</code> , <code>phase</code> , <code>wait_ctr</code> , <code>char_count</code> , <code>line1</code> , <code>line2</code> , <code>ready</code>
<code>tb/lcd_controller_tb.v</code>	<code>sim_verification</code>	Self-checking stream and timing test	<code>lcd_controller_tb</code> , <code>N</code> , <code>exp_data</code> , <code>exp_rs</code> , <code>negedge en checker</code> , <code>clear_us</code>

File	Component	Responsibility	Important symbols
constraints/lcd_controller.ucf	fpga_integration	Pin LOCs (template), I/O standard, clock period	TS_clk, NET entries for clk rst rs rw en data<7:0>
Makefile	sim_verification	Compile/run with Icarus, open GTKWave	sim, wave, clean
docs/images/fsm_diagram.svg	lcd_fsm (docs)	State diagram (matches RTL)	—
README.md	docs	User-facing guide	—

6. Relationship Map

```

tick_generator → lcd_fsm      : one-cycle tick paces every FSM step (50 µs default)
message_rom    → lcd_fsm      : ASCII bytes for WRITE1/WRITE2, indexed by char_count
lcd_fsm        → HD44780 LCD  : rs, rw(=0), en, data[7:0] (write-only, 8-bit)
lcd_fsm        → ready        : high in S_DONE
fpga_integration → lcd_controller : pins + 50 MHz PERIOD constraint (ISE only)
sim_verification → lcd_controller : instantiates DUT with compressed params; reads dut.CLEAR_TICKS,
dut.EN_US hierarchically

```

7. Main Workflows

W1 — Power-on display sequence (hardware/RTL)

1. rst deasserts; FSM is in S_POWER_UP.
2. Wait BOOT_TICKS ticks (≈20 ms at the defaults).
3. S_FUNCTION 0x38 → S_DISP_ON 0x0C → S_CLEAR 0x01 (each takes 2 ticks via pulse).
4. S_CLEAR_WAIT: wait CLEAR_TICKS ticks (≈2 ms).
5. S_ENTRY 0x06 → S_SET_LINE1 0x80.
6. S_WRITE1: rs=1, 11 bytes from line1, then char_count resets.
7. S_SET_LINE2 0xC0 → S_WRITE2: 12 bytes from line2.
8. S_DONE: EN held low, ready = 1. The sequence never runs again unless reset.

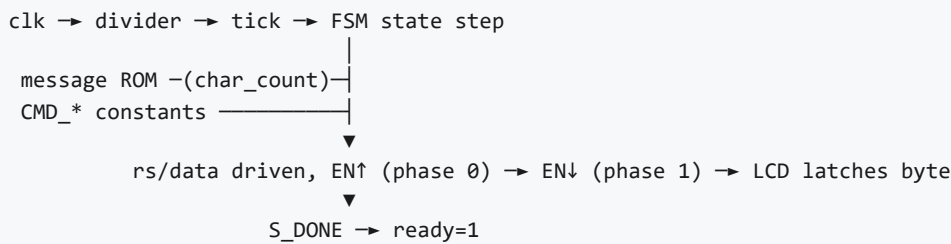
W2 — Simulation

1. make sim (or iverilog -g2012 ... && vvp lcd.out).
2. Release reset at 100 ns; the checker logs each byte on negedge en.
3. wait(ready), then +1000 ns.
4. Clear-delay check from the parameters; print ALL CHECKS PASSED or the failure count.
5. Optional: make wave opens lcd_sim.vcd in GTKWave.

W3 — FPGA build (ISE)

1. Replace every LOC in the UCF using the board schematic.
2. Synthesise and implement rt1/lcd_controller.v with the UCF in Xilinx ISE (Vivado does not support Spartan-6).
3. Generate the bitstream and program the board. Exact ISE project settings: UNKNOWN / NOT DOCUMENTED.

8. Data Flow



There is no input data path: the only inputs are clock and reset, and the output is fully determined by the parameters and the ROM.

9. Important Logic

Logic	What it does	Why	Where	Assumptions
Tick-paced FSM	All timing is a multiple of one <code>EN_US</code> tick	One clock-independent time base; changing <code>CLK_HZ</code> keeps the μ s timing	<code>TICKS</code> , <code>tick</code>	<code>CLK_HZ</code> is a multiple of 1 MHz (integer division)
<code>pulse(next)</code> task	Two-phase EN strobe for command bytes	Reuses one strobe pattern across all command states	<code>task_pulse</code>	Byte stays stable through both phases
<code>CLEAR_WAIT</code>	~2.05 ms wait after 0x01	Clear Display needs 1.52 ms; other commands need ~37 μ s	<code>S_CLEAR_WAIT</code> , <code>CLEAR_TICKS</code>	<code>EN_US</code> $\geq 37 \mu$ s so a single tick covers ordinary commands
Power-on wait	~20 ms before the first command	HD44780 needs >15 ms after Vdd	<code>S_POWER_UP</code> , <code>BOOT_TICKS</code>	Reset released after the LCD is powered
Write loop	Self-loop with <code>char_count++</code> , exits at <code>LENx-1</code>	Streams ASCII	<code>S_WRITE1</code> , <code>S_WRITE2</code>	<code>LENx</code> ≤ 16
Datasheet-check TB	Checks the delay from parameters, not sim time	TB uses compressed timing	TB final <code>initial</code>	Hierarchical refs <code>dut.CLEAR_TICKS</code> are supported by the simulator

10. Configuration

Setting	Location	Purpose	Values
<code>CLK_HZ</code>	<code>rtl/lcd_controller.v</code> parameter	Board clock frequency	default 50_000_000; TB uses 1_000_000
<code>EN_US</code>	same	Tick / EN settle window (μ s)	default 50; TB uses 2
<code>BOOT_TICKS</code>	localparam	Power-on wait	401 at defaults (≈ 20.05 ms)
<code>CLEAR_TICKS</code>	localparam	Clear Display wait	41 at defaults (≈ 2.05 ms); 1001 in TB (2002 μ s)
<code>CMD_*</code>	localparams	HD44780 commands	0x38, 0x0C, 0x01, 0x06, 0x80, 0xC0
<code>line1 / line2</code> , <code>LEN1 / LEN2</code>	<code>initial</code> block	Displayed text	"SALSABEEL K" (11), "SPARTAN6 LCD" (12)
<code>TS_clk</code>	UCF	Clock constraint	20 ns (50 MHz)
<code>LOC</code>	UCF	Pin assignment	placeholders , must be replaced

There are no secrets or environment variables in this repo.

11. Hardware Map

Hardware	Interface	Connected to	Purpose	Confidence
Spartan-6 FPGA	—	—	Runs <code>lcd_controller</code>	DOCUMENTED; exact device/package/board UNKNOWN
50 MHz oscillator	<code>clk</code>	FPGA pin (placeholder P123)	System clock	Frequency DOCUMENTED (UCF/param); pin NOT VERIFIED
Reset button/switch	<code>rst</code> (active high, PULLDOWN)	placeholder P124	Restart sequence	NOT VERIFIED
HD44780 16×2 LCD	8-bit parallel: <code>rs</code> , <code>rw</code> , <code>en</code> , <code>data[7:0]</code>	placeholders P131–P142	Display	Interface VERIFIED; wiring NOT VERIFIED FROM AVAILABLE SOURCES
LED (optional)	<code>ready</code>	commented out (P143)	Status	Optional
Power	LCD Vdd 5 V / logic 3.3 V	—	—	DOCUMENTED (UCF comment only)

12. Software Stack

- **Language:** Verilog (synthesizable RTL plus a behavioural TB)
- **Simulator:** Icarus Verilog (`iverilog -g2012, vvp`)
- **Waveform viewer:** GTKWave
- **Synthesis/implementation:** Xilinx ISE (required for Spartan-6); version UNKNOWN
- **Build:** GNU Make (`clean` uses `rm -f`, which needs a POSIX shell)
- **Database / runtime / OS / external services:** none

13. Important Commands

Purpose	Command	Source
Simulate	<code>make sim</code>	Makefile
Simulate (no make)	<code>iverilog -g2012 -o lcd.out rtl/lcd_controller.v tb/lcd_controller_tb.v && vvp lcd.out</code>	Makefile / README
Waveforms	<code>make wave (gtkwave lcd_sim.vcd)</code>	Makefile
Clean	<code>make clean</code>	Makefile
Synthesis	Xilinx ISE GUI; no script in the repo	README

14. Known Problems / Failure Modes

#	Problem	Cause	Effect	File	Workaround	Label
F1	Commands dropped after Clear	Next command issued during the 1.52 ms clear	Missing first characters of line 1, intermittent display	<code>rtl/lcd_controller.v</code>	<code>S_CLEAR_WAIT</code> (already fixed in current RTL)	DOCUMENTED

#	Problem	Cause	Effect	File	Workaround	Label
F2	Board does nothing	UCF LOCs are placeholders	Bitstream programs but drives the wrong pins	constraints/lcd_controller.ucf	Replace every LOC from the board schematic	DOCUMENTED
F3	TB doesn't check the final character	N = 28 but the DUT writes 29 bytes (5 cmd + 11 + 1 + 12); exp_data stops at "C"	The last "D" of line 2 is never verified; "28 bytes verified" can pass even if byte 29 is wrong	tb/lcd_controller_tb.v	Set N = 29 and add exp_data[28] = "D"	INFERRED (static reading; not executed)
F4	Possible HD44780 address-setup violation	rs / data change on the same clock edge that raises en (phase 0)	tAS (RS→E1, ~40 ns min) is ~0 ns apart from routing skew; may be marginal on some modules	rtl/lcd_controller.v (pulse, write states)	Present the byte one tick before EN1	INFERRED; original hardware worked, but the corrected RTL is untested on hardware
F5	Blank display / no contrast	5 V LCD Vdd/contrast not supplied	No characters	UCF comment	Provide 5 V Vdd	DOCUMENTED
F6	Wrong timing with a non-integer-MHz clock	CLK_HZ/1_000_000 uses integer division	Tick period shorter than intended	rtl/lcd_controller.v	Use a clock that is a multiple of 1 MHz	INFERRED
F7	Text longer than 16 characters	char_count is 4-bit; the display is 16 columns	Overflow / wrap	rtl/lcd_controller.v	Keep LENx ≤ 16	INFERRED

15. Testing

- **Framework:** a plain Verilog self-checking testbench (Icarus).
- **Location:** tb/lcd_controller_tb.v. **Command:** make sim.
- **Tested:** command order and values, RS for every byte (28 of them), and that the Clear Display delay is $\geq 1520 \mu\text{s}$ (checked from the parameters).
- **Untested:** the 29th byte (F3); power-on delay $\geq 15 \text{ ms}$ (not asserted); EN pulse width / setup times; behaviour on reset mid-sequence; synthesis in ISE; hardware run of the corrected RTL.
- **Run in this documentation pass:** NOT RUN. Icarus Verilog and make are not installed on the inspection machine.

16. Important Files To Learn First

1. rtl/lcd_controller.v: the entire design, covering timing, FSM, ROM and HD44780 commands.

2. `docs/images/fsm_diagram.svg`: the visual state flow; it matches the RTL states.
3. `tb/lcd_controller_tb.v`: the exact expected byte stream and the timing-check approach.
4. `constraints/lcd_controller.ucf`: how the design reaches the pins, and why the LOCs must change.
5. `README.md`: timing rationale and provenance.
6. `Makefile`: the simulation commands.

17. Learning Components

ID	Name	Purpose	Files	Prereqs	Related	Difficulty	Why it matters
<code>tick_generator</code>	Tick Generator	Clock → μ s time base	<code>rtl</code>	Verilog counters	<code>lcd_fsm</code>	Easy	Basis of every delay
<code>lcd_fsm</code>	LCD FSM	Init + write sequence	<code>rtl</code>	<code>tick_generator</code> , HD44780 basics	<code>message_rom</code> , <code>sim_verification</code>	Medium	Core behaviour
<code>message_rom</code>	Message ROM	Display strings	<code>rtl</code>	Verilog arrays	<code>lcd_fsm</code>	Easy	How content is changed
<code>sim_verification</code>	Simulation Verification	Self-checking TB	<code>tb</code> , <code>Makefile</code>	<code>lcd_fsm</code>	—	Medium	How correctness is proven
<code>fpga_integration</code>	FPGA Integration	UCF, ISE flow	<code>constraints</code>	—	<code>lcd_fsm</code>	Easy–Medium	Reaching real hardware

18. Stage 1 Learning Facts

1. The design drives an HD44780 16×2 LCD in 8-bit mode. — `rtl/lcd_controller.v` header
2. `rw` is tied to 0, so the busy flag is never read and all waits are fixed delays. — `assign rw = 1'b0`
3. `rs = 0` selects a command and `rs = 1` selects character data. — `rtl` ports
4. The HD44780 latches the bus on the falling edge of EN. — `rtl` header, TB `negedge en`
5. One tick = `EN_US` μ s; `TICKS = (CLK_HZ/1_000_000)*EN_US` clocks. — `rtl`
6. The default clock is 50 MHz and the default `EN_US` is 50 μ s. — `rtl` parameters, UCF `TS_clk`
7. The FSM has 11 states, from `S_POWER_UP` to `S_DONE`, in a 4-bit encoding. — `rtl`
8. The init order is 0x38 → 0x0C → 0x01 → (wait) → 0x06 → 0x80. — `rtl`, TB
9. 0x38 = 8-bit, 2-line, 5×8 font. 0x0C = display on, cursor off. 0x06 = increment, no shift. — `rtl` `CMD_*`
10. 0x80 and 0xC0 are the DDRAM start addresses of lines 1 and 2. — `rtl`
11. Power-on wait = `BOOT_TICKS = 2000/EN_US + 1` (401 ticks $\approx$ 20 ms; the datasheet needs >15 ms). — `rtl`, README
12. Clear Display takes 1.52 ms. `CLEAR_TICKS = 2000/EN_US + 1` (41 ticks $\approx$ 2.05 ms). — `rtl`, README
13. Without `CLEAR_WAIT`, the command after Clear is ignored and the first characters of line 1 go missing. — `rtl` comment, README
14. Each byte uses 2 ticks: phase 0 drives the byte with EN high, phase 1 drives EN low. — `pulse`, `phase`
15. `wait_ctr` is shared by `S_POWER_UP` and `S_CLEAR_WAIT`. — `rtl`
16. Line 1 is "SALSABEEL K" (11 chars); line 2 is "SPARTAN6 LCD" (12 chars). — `rtl` ROM
17. `ready` is high only in `S_DONE`. — `assign ready`
18. The TB runs with `CLK_HZ = 1 MHz`, `EN_US = 2` to compress time. — TB
19. The TB checks Clear timing from `dut.CLEAR_TICKS * dut.EN_US`, not from simulation time. — TB
20. A Spartan-6 bitstream requires Xilinx ISE; Vivado does not support Spartan-6. — README
21. The UCF LOCs are placeholders that must be replaced. — UCF header
22. LCD outputs use LVCMOS33, DRIVE 8, SLEW SLOW to tolerate ribbon cables. — UCF

23. Reset is synchronous and active high, and returns the FSM to `S_POWER_UP`. — rtl
24. The current RTL is a reconstruction of a lost original and has not been re-run on hardware. — README Provenance

19. Stage 2 Candidates

lcd_fsm

- **Why:** sequencing, two-phase strobes and datasheet timing all meet here.
- **Files / symbols:** `rtl/lcd_controller.v`: `state`, `phase`, `pulse`, `wait_ctr`, `char_count`, `S_CLEAR_WAIT`.
- **Prediction questions:** how many ticks from reset to `ready`; what `data` / `en` are on a given tick; timing at a different `EN_US`.
- **Failure questions:** removing `S_CLEAR_WAIT`; `EN_US < 37`; `LEN1 > 16`; reset mid-write.
- **Modification questions:** add a third command (e.g. cursor on); make the message loop/refresh; add 4-bit mode; present the byte a tick before `EN↑` (F4).

tick_generator

- **Why:** parameter arithmetic and integer-division pitfalls.
- **Symbols:** `TICKS`, `div`, `$clog2`, `tick`.
- **Prediction:** `TICKS` and the `div` width for given params. **Failure:** `CLK_HZ = 33.333 MHz` (F6). **Modification:** a ns-resolution divider.

sim_verification

- **Why:** self-checking TB design and compressed timing.
- **Symbols:** `N`, `exp_data`, `exp_rs`, `negedge en` checker, `clear_us`.
- **Prediction:** what the TB prints if `CLEAR_TICKS` is halved. **Failure:** the missing 29th byte (F3). **Modification:** add a power-on-delay check and `EN` pulse-width assertions.

20. Ambiguities and Verification Required

Issue	Sources	Confidence	What to check
Corrected RTL never run on hardware	README Provenance	DOCUMENTED	Rebuild in ISE and test on the board
Board model, FPGA part/package, ISE version	—	UNKNOWN	Project report / board docs
Real pin wiring	UCF (placeholders)	NOT VERIFIED	Board schematic
Upstream origin and license of the adapted driver	README, rtl header	UNKNOWN	Identify the source; confirm MIT relicensing is compatible
"~30x" (rtl comment) vs "~40x" (README, SVG) for the Clear/normal command ratio	rtl line ~60; README; svg	CONFLICT (cosmetic)	1.52 ms / 37 μ s $\approx$ 41x, so the README is correct
README delay table gives lower bounds	README vs FSM	INFERRED	Actual <code>EN↓</code> →next <code>EN↑</code> after Clear $\approx$ <code>CLEAR_TICKS+1</code> ticks ($\approx$ 2.1 ms); power-on $\approx$ <code>BOOT_TICKS+1</code> ticks. Both still exceed the datasheet
TB "28 bytes verified" output	TB, README	INFERRED	The DUT emits 29 bytes; confirm by simulation (F3)
tAS setup margin	rtl	INFERRED	Scope RS vs <code>EN↑</code> on hardware (F4)
<code>\$clog2</code> in a port-adjacent reg declaration under XST	rtl	UNKNOWN	Confirm ISE synthesis is clean

Issue	Sources	Confidence	What to check
Board photos are from the original (lost) RTL	README	DOCUMENTED	They are not evidence for the current RTL

21. Project Snapshot

```

PROJECT: Verilog-LCD-Driver-Spartan6_FPGA
TYPE: FPGA RTL design (HD44780 LCD driver)
LANGUAGES: Verilog
FRAMEWORKS: none (tools: Icarus Verilog, GTKWave, Xilinx ISE, Make)
MAIN_COMPONENTS: tick_generator, lcd_fsm, message_rom, sim_verification, fpga_integration
ENTRY_POINTS: rtl/lcd_controller.v (module lcd_controller, top); tb/lcd_controller_tb.v (sim top)
MAIN_WORKFLOWS: power-on display sequence; self-checking simulation; ISE build with UCF
HARDWARE: Spartan-6 FPGA board, 50 MHz clock, HD44780 16x2 LCD (8-bit parallel)
EXTERNAL_SERVICES: none
DATABASE: none
TESTING: self-checking Verilog TB via `make sim` (28/29 bytes checked + Clear delay)
STATUS: simulation passing (documented); corrected RTL not hardware-verified

```

Current Project Snapshot

- **Last inspected:** 2026-10-01
- **Git:** branch `main`, commit `c81ea91`, clean before documentation; tracks `origin/main`; no tags
- **Recently changed important files (2026-08-24):**
 - `tb/lcd_controller_tb.v`: rewritten as self-checking (`c81ea91`)
 - `rtl/lcd_controller.v`: reconstruction plus `CLEAR_WAIT` fix (`bca1421`)
 - `docs/images/fsm_diagram.svg/.png`: added (`e330152`)
 - `constraints/lcd_controller.ucf`: created (`6333b29`)
 - `Makefile`, `README.md`: added/reworked (`bc66eaa`)
- **Initial commit:** `9cbae80` (2026-07-19)
- **Sensitive files:** none found